

DataUnderstandAgent 关键失效模式复盘

本文复盘 run `20260505T131439Z` 中 4 道错题，目标不是立刻给出改法，而是把“DataUnderstandAgent 为什么没有真正帮到下游模型”讲清楚。

这 4 道错题是：

Task	问题类型	Gold	Prediction	表面症状
<code>task_344</code>	医疗阈值 + 患者聚合	<code>4</code>	<code>0</code>	找到字段，但关键阈值未知
<code>task_379</code>	文档标签 + CSV 原子表	<code>element</code> 的若干值	输出大量逐分子元素	handoff fallback，缺可执行 contract
<code>task_396</code>	文档里隐含超级英雄表	<code>54.83870967741935</code>	无答案	handoff complete 但实际不可执行
<code>task_420</code>	CSV + SQLite join	<code>100.0</code>	<code>99.97297662478043</code>	join key 选错

1. 先建立一个判断标准

DataUnderstandAgent 的职责不是“回答题目”，而是给下游 answer agent 一个可执行的理解交接。

一个真正有用的 handoff 至少应该说清楚：

- 答案列是什么：**输出列名、是否只需要一个数、是否需要列表。
- 从哪里取行：**row source 是哪个表、文档、数据库，行粒度是什么。
- 怎么过滤：**每个自然语言条件对应哪个字段、哪个值、哪个比较方式。
- 怎么 join：**join key 是哪个，为什么选它，而不是另一个同名字段。
- 怎么聚合：**count、percentage、distinct、denominator 的范围。
- 哪些信息还没有被解决：**如果没有阈值、没有 join、没有 row source，就不能把 handoff 标成可信完成。

所以我们看错题时，不要只看 `handoff_status` 是不是 `complete`。更重要的是看：

- `answer_contract.filters` 是否真的可执行；
- `row_source` 是否非空；
- `join_policy` 和 join path 是否能落到具体字段；
- `remaining_uncertainties` 里有没有必须解决才能计算的内容；
- 下游是否因为 handoff 的模糊信息开始猜。

2. 失效模式一：字段找到了，但“业务阈值”没有找到

对应任务：`task_344`

问题：

Among the male patients who have a normal level of white blood cells, how many of them have an abnormal fibrinogen level?

gold 是:

```
COUNT(DISTINCT T1.ID)
4
```

prediction 是:

```
count
0
```

DataUnderstandAgent 这题做对了一部分: 它能把实体和字段对应起来。

handoff 里写的是:

```
patients → csv/Laboratory.csv.ID, patient_sex.csv.ID
male → patient_sex.csv.SEX
white blood cells normal → csv/Laboratory.csv.WBC
fibrinogen abnormal → csv/Laboratory.csv.FG
join: csv/Laboratory.csv.ID ⇒ patient_sex.csv.ID
```

这些都是有价值的信息。问题出在两个过滤条件:

```
csv/Laboratory.csv.WBC within normal range (threshold unknown)
csv/Laboratory.csv.FG abnormal (threshold unknown)
```

这两个条件不是“稍微不确定”，而是计算题目的核心。没有 WBC normal 的上下界，也没有 FG abnormal 的判断规则，下游 answer agent 只能猜。

实际 trace 里下游开始尝试各种医学常识和分布猜测:

- WBC normal 用 4.0-11.0 ;
- FG 先尝试 ≥ 75 ;
- 后来又尝试 20-40 作为 normal ;
- 再尝试均值加减 2 倍标准差 ;
- 再尝试 IQR outlier。

这些都不是来自数据上下文的确定规则，所以结果跑到了 0。

这说明第一个问题:

DataUnderstandAgent 现在会把“阈值未知”原样交给下游，但又把 handoff 标成 **complete**。

这会让下游误以为“字段层面已经解决，只剩执行”，实际却是“题目的判定规则没有解决”。这种 handoff 看起来有帮助，实际上把风险后移了。

更好的 handoff 应该是下面这种语义：

```
status: blocked_or_incomplete
blocking_uncertainties:
- WBC normal range is required but not found in knowledge/context.
- FG abnormal range is required but not found in knowledge/context.
do_not_guess:
- Do not infer thresholds from distribution unless a fallback policy
  explicitly allows it.
```

也就是说，这类题不是缺 SQL 能力，而是缺 **reference range / threshold discovery** 能力，以及更严格的 complete 判定。

3. 失效模式二：文档里有实体属性，但工具只把它当长文本

对应任务： **task_379**

问题：

Tally the toxicology element of the 4th atom of each molecule that was carcinogenic.

gold 输出列是：

```
element
c
br
cl
s
o
```

prediction 前几行是：

```
element
c
c
c
c
br
```

这个题的上下文很特殊：

- `csv/atom.csv` 有结构化字段: `atom_id,molecule_id,element`
- `doc/molecule.md` 里用自然语言描述哪些 `TRxxx` molecule 是 carcinogenic
- `knowledge.md` 说 Molecule 有 `molecule_id` 和 `label`
- 但实际 context 里没有 `csv/molecule.csv`

DataUnderstandAgent 发现了这个断层,但没有能力补上它。

handoff 最终是 fallback:

```
handoff_status: fallback
answer_columns: []
filters: []
metric_operation: unknown
row_source: ""
join_policy: unknown
```

validation error 是:

```
Contract uses rejected fields: ['csv/atom.csv.molecule_id']
```

这个错误很关键。DataUnderstandAgent 认为:

- `atom.csv.molecule_id` 只是 atom 到 molecule 的链接;
- 它不包含 carcinogenic 标签;
- 所以不能用它表达 carcinogenic filter。

这个判断一半对,一半错。

对的部分是: `molecule_id` 本身确实不是标签。

错的部分是: `molecule_id` 正是连接 `atom.csv` 和文档中 `TRxxx → carcinogenic` 标签的唯一钥匙。没有结构化 molecule 表时,正确做法不是拒绝 `molecule_id`,而是构造一个“文档抽取出来的虚拟表”:

```
doc/molecule.md extracted table:
- molecule_id
- carcinogenic_label
```

然后再和 `atom.csv.molecule_id` 连接。

当前工具只能做:

- `lookup_knowledge`: 搜到文档片段;
- `search_semantic_index`: 知道 `atom_id/molecule_id/element` 相关;
- `get_asset_schema(csv/molecule.csv)`: 返回空;

- `find_join_paths(atom.csv, molecule.csv)`：找不到。

它不能做：

- 从文档里抽取所有 `TRxxx`；
- 判断每个 `TRxxx` 的最终 carcinogenic 状态；
- 把抽取结果暴露成可 join 的结构；
- 验证 `atom_id` 的 `_4` 后缀是否稳定表示第 4 个 atom。

下游 answer agent 后来自己写 Python 解析 `doc/molecule.md`，并用 `atom_id.endswith('_4')` 找第 4 个 atom。这个方向接近正确，但因为 DataUnderstand 没有提前定好输出 grain 和“tally”的含义，下游输出了 99 行，而 gold 只需要去重后的元素集合。

这里暴露了第二个问题：

DataUnderstandAgent 对“文档承载结构化事实”的场景没有足够工具，只能指出风险，不能把风险转成可执行结构。

这一类题需要的不是再多搜几次文档，而是一个明确的工具能力：

```
extract_document_entities(  
    document="doc/molecule.md",  
    entity_key_pattern="TR\\d+",  
    fields=["carcinogenic_label"],  
    correction_policy="use final corrected status"  
)
```

以及一个字段模式验证：

```
verify_position_from_id(  
    field="csv/atom.csv.atom_id",  
    pattern="{molecule_id}_{position}",  
    requested_position=4  
)
```

4. 失效模式三：handoff 标 complete，但 contract 实际不可执行

对应任务： `task_396`

问题：

In superheroes with height between 150 to 180, what is the percentage of heroes published by Marvel Comics?

gold 是：

```
54.83870967741935
```

prediction 不存在，因为下游跑满 `max_steps=100` 没有提交答案。

这题的数据上下文也很典型：

- `json/publisher.json` 是结构化 publisher 维表；
- `doc/superhero.md` 是一份很长的文档；
- `knowledge.md` 明确说 superhero 应该有 `height_cm`、`publisher_id` 等字段；
- 但实际没有 `superhero.csv` 或 `superhero.db`。

DataUnderstandAgent 的 handoff 状态是：

```
handoff_status: complete  
validation_errors: []
```

但 answer contract 是：

```
filters: []  
metric_operation: unknown  
metric_fields: []  
row_source: ""  
join_policy: unknown  
output_grain: superhero
```

这就是一个“假 complete”。

它虽然写了答案列 `percentage`，但没有告诉下游：

- 从哪里读 superhero 行；
- height 在哪里；
- publisher code 在哪里；
- publisher code 如何 join 到 `publisher.json.records.id`；
- denominator 是哪些 superhero；
- numerator 是哪些 Marvel superhero。

更糟的是，handoff 的 `brief_markdown` 最后还写：

```
Use this handoff as trusted guidance; call tools to compute the requested  
result.
```

这会把下游推进一个很尴尬的状态：它被告知 handoff 可信，但 handoff 又没有可执行信息。于是下游开始自己解析超长文档，反复写正则，最后陷入循环。

trace 中能看到下游多次尝试：

- 搜 height；
- 搜 publisher；
- 搜同时含 height 和 publisher 的窗口；
- 解析 superhero entry；
- 但 height 信息和 publisher 信息在文档不同章节，简单上下文窗口很难同时抓住。

这说明第三个问题：

complete 的判定没有检查 answer contract 是否可执行。

一个 handoff 即使没有 validation error，也不应该 complete，只要出现下面任意情况：

```
row_source == ""
filters == []
metric_operation == "unknown"
join_policy == "unknown"
question filters were not grounded
blocking uncertainties remain
```

对于 **task_396**，正确状态应该是类似：

```
status: incomplete
reason:
- superhero entity exists only in doc/superhero.md
- height_cm and publisher_id must be extracted from document sections
- publisher_id → publisher_name mapping exists in json/publisher.json
needed_tool:
- document_entity_table_extraction keyed by superhero id
```

这类问题的本质不是模型不会读，而是工具没有把“长文档中的多章节实体属性”整理成表。

对于这类文档，正确抽取应该按实体 ID 聚合多段信息：

superhero_id	superhero_name	height_cm	publisher_id
7	Absorbing Man	193	...
26	Angel Dust	165	...
...			

然后再执行：

```
denominator: superheroes where 150 ≤ height_cm ≤ 180
numerator: denominator and publisher_name = 'Marvel Comics'
percentage: numerator * 100 / denominator
```

5. 失效模式四：多个 join candidate 时，只看字段名，不做数据级验证

对应任务： **task_420**

问题：

What percentage of cards with format commander and legal status do not have a content warning?

gold 是：

100.0

prediction 是：

99.97297662478043

这题 DataUnderstandAgent 表面上做得很好：

```
cards → db/cards.db.cards.id, db/cards.db.cards.uuid
format commander → csv/legalities.csv.format
legal status → csv/legalities.csv.status
no content warning → db/cards.db.cards.hasContentWarning
```

但它选错了 join：

csv/legalities.csv.id ⇒ db/cards.db.cards.id

实际数据里， **legalities.csv** 同时有：

id, format, status, uuid

cards.db.cards 也同时有：


```
id, uuid
```

所以有两个候选 join:

1. `legalities.id = cards.id`
2. `legalities.uuid = cards.uuid`

DataUnderstandAgent 选择了 `id`, 因为字段名相同, 且 semantic catalog 里 `id` 和 `uuid` 都只是 medium confidence。

但数据级验证会马上发现差异:

```
Using id join:
  matched denominator = 7401
  no warning numerator = 7399
  percentage = 99.97297662478043
  missing legalities rows = 47843
```

```
Using uuid join:
  matched denominator = 55235
  no warning numerator = 55235
  percentage = 100.0
  missing legalities rows = 9
```

gold 是 `100.0`, 说明应该用 `uuid` join。

这说明第四个问题:

join path 不能只靠字段名和 schema 样本, 必须做 value overlap / cardinality 验证。

特别是这类库里, `id` 很可能是“当前表自己的行号”, 而 `uuid` 才是跨表实体 ID。`legalities.id` 是 legality row id, 不是 card id; `legalities.uuid` 才能指向 `cards.uuid`。

更可靠的 join handoff 应该包含这样的证据:

```
candidate joins:
- legalities.id → cards.id:
  overlap/match rate low for commander+Legal scope
  likely row-id-to-row-id false join
- legalities.uuid → cards.uuid:
  overlap/match rate high
  preserves card identity
selected join:
- legalities.uuid → cards.uuid
```

如果 DataUnderstandAgent 当时能调用一个 join validation 工具，下游不会算错。

6. 失效模式五：工具反复调用，但信息类型不匹配

这 4 道题里，DataUnderstandAgent 不是没有调用工具，而是工具返回的信息类型经常和问题需要的信息类型不匹配。

task_344

它调用了：

```
lookup_knowledge: 8 次  
get_asset_schema: 4 次  
search_semantic_index: 2 次  
find_join_paths: 2 次
```

但问题需要的是 WBC 和 FG 的 reference range。schema 和 semantic search 只能告诉你字段存在，不能告诉你 normal/abnormal 的数值定义。

task_379

它调用了：

```
lookup_knowledge: 6 次  
get_asset_schema: 4 次  
search_semantic_index: 6 次  
find_join_paths: 4 次
```

但问题需要的是“把 doc/molecule.md 抽成 molecule label 表”。现有工具只能搜片段，不能产出结构化抽取结果。

task_396

它调用了：

```
search_semantic_index: 7 次  
get_asset_schema: 7 次  
lookup_knowledge: 6 次  
find_join_paths: 6 次
```

但问题需要的是“按 superhero id 汇总文档中分散的 height 和 publisher 属性”。现有工具没有实体级文档抽取和跨章节合并能力。

task_420

它调用了 schema 和 semantic search，但没有做 join 候选实测。因此它知道有 `id` 和 `uuid`，却不能判断哪个才是对的。

所以这不是单纯的 prompt 问题。prompt 让模型“多想想”可能会缓解一部分，但核心瓶颈是工具能力没有覆盖这些信息形态。

7. 从浅到深总结这些问题

第一层：字段级理解够了，但规则级理解不够

`task_344` 说明 agent 可以找到字段，却找不到判定规则。

字段映射：

```
WBC → csv/Laboratory.csv.WBC
FG → csv/Laboratory.csv.FG
```

不等于过滤条件可执行：

```
WBC normal → ? ≤ WBC ≤ ?
FG abnormal → FG < ? or FG > ?
```

第二层：schema 里没有表，不代表事实不存在

`task_379` 和 `task_396` 都是这个问题。

事实在文档里，但不是表：

```
TR391 is carcinogenic
hero 26 has height 165
hero 26 has publisher_id ...
```

当前 DataUnderstandAgent 对这种事实的处理方式是“搜片段”，不是“抽成表”。

第三层：join 不是字段同名游戏

`task_420` 说明 join key 必须通过数据验证。

同名字段可能是错的：

```
legalities.id ≠ cards.id
```

看起来更复杂的字段反而是对的：

```
legalities.uuid == cards.uuid
```

第四层：handoff 状态会影响下游行为

task_396 最危险的点不是它没找到信息，而是它没找到信息还标了 complete。

下游看到：

```
Use this handoff as trusted guidance
```

但 contract 实际是：

```
filters: []  
row_source: ""  
metric_operation: unknown  
join_policy: unknown
```

这会让下游陷入“我应该能算，但不知道怎么算”的循环。

第五层：缺的不是一个工具，而是一组不同层次的工具

这些错题分别需要：

- threshold/reference range lookup；
- document-to-table extraction；
- ID pattern / ordinal position validation；
- join candidate value-overlap validation；
- handoff contract executability validation。

这些工具解决的是不同层级的问题，不能互相替代。

8. 一个更实用的读 trace 方法

以后看 DataUnderstandAgent 的效果，可以按这个顺序读：

1. 先看 gold/pred 差异

是值错、列错、没提交，还是冗余？

2. 看 answer contract 是否完整

如果 **filters=[]**、**row_source=""**、**metric_operation=unknown**，handoff_status 再好看也没有用。

3. 看 **remaining_uncertainties** 是否是 **blocking**

如果不确定的是“阈值”“join key”“denominator”，那就是 blocking，不应该 complete。

4. 看工具调用是否在重复同一种搜索

如果 lookup/search 重复很多次但仍然没有新字段，说明缺的是新工具，不是多搜一次。

5. 看下游是否开始猜

下游一旦开始“用常识猜阈值”“用正则碰文档”“试多个 join”，说明 handoff 没有真正完成任务。

9. 这次 run 给出的核心判断

当前 DataUnderstandAgent 的有效提升有限，主要不是因为模型完全不会分析，而是因为它能提供的信息停在了较浅层：

- 它擅长发现候选字段；
- 它能列出风险；
- 它能做简单 join path；
- 但它不擅长把风险解决掉；
- 更不擅长把文档事实、阈值规则、join 证据转成可执行 contract。

所以后续如果要改，优先级应该不是“让 handoff 更长”，而是让 handoff 更可执行。

最重要的质量门槛是：

complete handoff = 下游可以直接计算，不需要猜核心规则

如果做不到，就应该明确标记为 incomplete/fallback，并告诉下游缺什么、不要猜什么、需要什么工具来补。